MicroROM-NPU

Microroutine Specification: Routine 0, LB and LW

Separate working document
This document collects the microroutines formulated so far for the proposed 2D vertical MicroROM. It explains the intent, step sequencing, relevant control fields, packed words where derived, example behavior, and checks still required before the ROM contents are frozen.

Specification status: DRAFT FOR VALIDATION. The symbolic sequences and packed words are derived from the current 48-bit field definitions. They have not yet been checked against executable RTL or simulation. Do not treat them as proven-correct silicon-ready contents.

Scope of this document

Item	Current project decision
MicroROM organization	2D logical constant ROM: 10 routine IDs x 32 local steps x 48 bits per word.
Architectural PC	Exactly one PC register. Fetch advances it; a taken branch/jump can replace it later.
Instruction subset	LB, LW, SW, ADD, ADDI, BNE, BGE and JAL (not full RV32I compliance).
Microsequencer	Every transition is completion-gated. An incomplete operation holds the current routine ID and local step.
Routines documented here	Routine 0: fetch/decode/dispatch; Routine 1: LB; Routine 2: LW.

Version note

Prepared from the current design decisions as of 14 October 2026. Control semantics or packed words may be revised if legality checks or simulation reveal a conflict. Later microroutines must follow the same contracts.

1. Shared architecture and encoding

The MicroROM stores fixed control words. Values that vary from instruction to instruction are taken from the instruction register (IR) at run time. For example, RS1 and RD are selector codes that resolve to the register numbers encoded in IR; they are not extra physical registers.

48-bit microinstruction layout

Field	Bit positions	Width	Current meaning
UOP	[47:43]	5	Datapath or special operation
SRC1_SEL	[42:37]	6	First source selector
SRC2_SEL	[36:31]	6	Second source selector
DST_SEL	[30:25]	6	Destination selector
NEGATE	[24]	1	Negate SRC2 for ADD only
IMM_SEL	[23:21]	3	Select immediate or small constant
MEM_CTRL	[20:18]	3	Memory transaction class
COND	[17:15]	3	Registered-flag condition
NEXT	[14:12]	3	Microsequencer transition
NEXT_ADDR	[11:7]	5	Local target for JUMP/COND
CALL_MUL	[6]	1	Call multiplication subroutine when legal
PC_CTRL	[5:4]	2	Hold, advance, or load PC target
FLAGS_CTRL	[3:2]	2	Preserve, compare, or clear flags
RESERVED	[1:0]	2	Must be 00

Relevant operation codes

UOP bits	Name	Semantics relevant here
00000	NOP	No datapath operation/write. Independent MEM_CTRL, PC_CTRL and NEXT controls may still act.
00001	MOV	DST <- SRC1.
00011	ADDI	DST <- SRC1 + selected sign-extended I-type immediate.
01000	LOAD	Blocking data-memory load into DST; exact behavior selected by MEM_CTRL.
01010	DECODE	Decode IR and expose validity plus selected routine ID.
01101	FETCH	Special operation: MAR <- PC. Selector fields are unused and canonically zero.

Relevant selector codes

Selector bits	Decimal	Name / behavior
000000	0	x0: reads zero; writes ignored.
100110	38	MDR: memory data register.
100111	39	MAR: memory address register.
101001	41	RS1: dynamically selects x[IR.rs1].
101011	43	RD: dynamically selects x[IR.rd].
101100	44	IR: instruction register.

Important: selector codes RS1 and RD are interpreted dynamically by the register-selection logic. The MicroROM does not change when IR contains different register numbers.

2. Routine 0 - instruction fetch, decode and dispatch

Routine ID: 0000. Current plan: local steps 0 through 5. All instructions in the present subset are 32 bits, so the fetch-time PC advancement is +4 bytes.

Step sequence

Step	UOP	Other active control	Behavior and completion condition
0	FETCH (01101)	NEXT=SEQ	MAR <- PC. Complete when MAR has captured PC. This preserves the fetched instruction address in MAR.
1	NOP (00000)	MEM_CTRL=100; NEXT=SEQ	Request instruction-memory read. Wait for request acceptance and response. On response capture, MDR <- instruction data. Complete only after data is captured.
2	MOV (00001)	SRC1=MDR; DST=IR; NEXT=SEQ	IR <- MDR. Complete when IR captures the fetched word.
3	NOP (00000)	PC_CTRL=01; NEXT=SEQ	PC <- PC + instr_len; instr_len is 4 in the current 32-bit-only design. Commit exactly once.
4	DECODE (01010)	NEXT=SEQ	Decode IR, validate the supported encoding, expose instruction operands/immediate and routine ID.
5	NOP (00000)	NEXT=DISPATCH	For valid instruction, enter its execution routine at step 0. Invalid encoding enters illegal-instruction error state instead.

Packed 48-bit words from the current field assignments

Step	Packed word	Interpretation
0	48'h680000000000	FETCH; MAR <- PC.
1	48'h000000100000	NOP plus MEM_CTRL=100 instruction-read transaction.
2	48'h0CC058000000	MOV MDR to IR.
3	48'h000000000010	NOP plus PC_CTRL=01.
4	48'h500000000000	DECODE.
5	48'h000000003000	NOP plus NEXT=DISPATCH.

Handshake and stall contract

At step 1, the instruction-read request must be held until accepted. After acceptance, the controller waits for response-valid and captures the returned instruction into MDR exactly once. The microsequencer remains at (routine 0, step 1) until the complete transaction succeeds; it must not reissue the request during a response wait. The interface must also handle a request acceptance and response arriving in the same cycle if the selected memory protocol permits it.

Single-PC rule: fetch is the only step in Routine 0 that advances PC. MAR retains the current instruction address until overwritten. Branch and jump routines may use MAR as the instruction address, but must calculate their target before reusing MAR for a data-memory address.

Dispatch targets: LB=0001, LW=0010, SW=0011, ADD=0100, ADDI=0101, BNE=0110, BGE=0111, JAL=1000. An unsupported instruction enters the illegal-instruction error state instead of dispatching.

3. Routine 1 - LB (Load Byte)

Routine ID: 0001. This routine loads one byte from the effective address and sign-extends that byte to 32 bits. It uses the same three ROM words for every valid rs1, rd and signed 12-bit immediate.

Instruction-level behavior

```
x[rd] <- SignExtend32(MemoryByte(x[rs1] + SignExtend32(imm[11:0])))
```

Step sequence

Step	UOP	Key controls	Behavior and completion condition
0	ADDI (00011)	SRC1=RS1; DST=MAR; IMM_SEL=010	MAR <- x[IR.rs1] + SignExtend32(IR[31:20]). The I-type immediate is signed.
1	LOAD (01000)	DST=MDR; MEM_CTRL=001	Read one byte at MAR, sign-extend to 32 bits, and capture in MDR. Hold this step until the memory transaction completes.
2	MOV (00001)	SRC1=MDR; DST=RD; NEXT=RESTART	x[IR.rd] <- MDR. Ignore writes to x0. Restart Routine 0 at step 0 only after writeback completes.

Packed 48-bit words

Step	Packed word	Key interpretation
0	48'h1D204E400000	ADDI using dynamic RS1 plus I-type immediate; destination MAR.
1	48'h40004C040000	LOAD to MDR; signed byte-read control MEM_CTRL=001.
2	48'h0CC056005000	MOV MDR to dynamic RD; NEXT=RESTART.

Example execution

```
Instruction: LB x5, -4(x9)
Assumption: x9 = 0x00002000; memory byte at 0x00001FFC = 0xFE.
```

Step	Result
0	MAR = 0x00001FFC (0x2000 - 4).
1	Byte 0xFE has its top bit set. Sign extension gives MDR = 0xFFFFFFFE.
2	x5 = 0xFFFFFFFE; microsequencer restarts Routine 0, step 0.

Operand variability

The bits in the MicroROM do not encode a particular x-register number. RS1 resolves to IR[19:15], RD resolves to IR[11:7], and IMM_SEL=010 selects the signed I-type immediate from IR[31:20]. This is why a single microroutine handles all encoded register and immediate values.

Required LOAD contract: MEM_CTRL=001 selects a signed byte read. The returned byte is sign-extended to 32 bits and written to MDR once, after a successful response. The memory controller must not repeat the transaction while the microsequencer is stalled.

4. Routine 2 - LW (Load Word)

Routine ID: 0010. This routine loads a 32-bit word from the effective address. The value is already 32 bits, so no additional sign extension is required.

Instruction-level behavior

```
x[rd] <- MemoryWord32(x[rs1] + SignExtend32(imm[11:0]))
```

Step sequence

Step	UOP	Key controls	Behavior and completion condition
0	ADDI (00011)	SRC1=RS1; DST=MAR; IMM_SEL=010	MAR <- x[IR.rs1] + SignExtend32(IR[31:20]).
1	LOAD (01000)	DST=MDR; MEM_CTRL=010	Read the 32-bit word at MAR and capture in MDR. Hold this step until the memory response is accepted.
2	MOV (00001)	SRC1=MDR; DST=RD; NEXT=RESTART	x[IR.rd] <- MDR. Ignore writes to x0. Restart Routine 0, step 0 after writeback completes.

Packed 48-bit words

Step	Packed word	Key interpretation
0	48'h1D204E400000	Same effective-address calculation as LB.
1	48'h40004C080000	LOAD to MDR; word-read control MEM_CTRL=010.
2	48'h0CC056005000	Same dynamic destination writeback and restart as LB.

Example execution

```
Instruction: LW x12, 8(x2)
Assumption: x2 = 0x00003000; 32-bit word at 0x00003008 = 0x89ABCDEF.
```

Step	Result
0	MAR = 0x00003008.
1	MDR = 0x89ABCDEF after the word-read response completes.
2	x12 = 0x89ABCDEF; microsequencer restarts Routine 0, step 0.

Alignment policy to decide

The current routine assumes the memory system can complete the requested 32-bit word read. Before implementation freeze, explicitly choose the behavior for misaligned LW addresses. A simple educational implementation can support naturally aligned word accesses and route unsupported misaligned accesses to a defined data-access error state; that is a project policy to approve, not a completed design decision.

Difference from LB: only the memory operation changes: MEM_CTRL=001 reads a byte and sign-extends it; MEM_CTRL=010 reads a complete 32-bit word. The effective-address and architectural writeback steps are identical.

5. Cross-routine checks and freeze checklist

The three routines are currently coherent at the symbolic level, but the following shared contracts must be implemented consistently before declaring the ROM frozen.

Behavioral invariants

Invariant	Required behavior
Completion-gated sequencing	For any microinstruction, apply NEXT only after the active operation completes. While incomplete, hold routine ID and local micro-step.
No duplicate memory operation	Track whether a request was accepted; do not issue the same read again while waiting for its response.
Single PC	Routine 0 advances PC once after instruction capture. LB/LW do not modify PC. Exactly one PC action may commit per clock.
Dynamic operands	RS1 and RD select registers using the fields in IR at run time; no per-register microroutines.
x0 behavior	Reads return zero and writes are ignored.
Reserved bits	RESERVED[1:0] must be 00; illegal words suppress side effects and enter the microinstruction error state.
Instruction errors	An unsupported instruction must not dispatch to routine 0 or any execution routine; signal illegal_instruction.
Data access errors	A memory access fault must suppress LB/LW writeback and normal restart. Its exact interface and error-state policy remain to be specified.

Recommended verification cases

Routine	Test case	Expected result
R0	Request not accepted for several cycles	Remain at step 1; hold request; no duplicate transaction.
R0	Request accepted, response delayed	Deassert request after acceptance; remain at step 1 until response.
R0	Valid LB/LW/other selected instruction	Capture IR, advance PC once, dispatch to correct routine ID.
R0	Unsupported instruction encoding	Enter illegal-instruction error state; do not dispatch.
LB	Negative immediate and positive immediate	Effective address uses signed 12-bit immediate.
LB	Memory byte 0x7F then 0x80 then 0xFE	Results 0x0000007F, 0xFFFFFF80, and 0xFFFFFFFE respectively.
LB	rd = x0	Memory operation executes but architectural write is ignored.
LW	Distinct rs1, rd and immediate values	Correct effective address and dynamic destination.
LW	Word data 0x89ABCDEF	Write exactly 0x89ABCDEF to rd.
LB/LW	Memory response delayed	No early writeback or routine restart.

Freeze criteria

Check	Current status
48-bit field boundaries and selector values	Defined in this working spec.
Packed words for R0, LB and LW	Derived from current encodings; not RTL-simulated.
Microinstruction legality table for all field combinations	Still requires full consistency pass.
Memory request/response state behavior	Contract described; interface signals and fault path still need definition.
RESTART transition semantics	Proposed: routine 0, local step 0; validate in sequencer specification.
Misaligned LW policy	Not finalized.
Full ROM rows 0-9 and default/unused entries	Not yet populated or frozen.

Next planned work: continue with Routine 3 (SW), then ADD, ADDI, BNE, BGE, JAL and the repeated-add multiplication subroutine. After every routine is specified, run a whole-ROM consistency check and verify the encoded words before freezing the complete 2D ROM.